

S t a t e

State Pattern

Behavior changes with state

DEFINITION

Allow an object to alter its behavior when its internal state changes — it appears to change class.

State gives an object one class per mode: the Context holds a current State and delegates every request to it, so the same call behaves differently as the state changes — the object appears to change class. Crucially, the states drive their own transitions, swapping the Context to the next mode.

WHY IT MATTERS

A single object whose behavior depends on a status field grows sprawling switch ladders repeated across methods, easy to get inconsistent. State localizes each mode's behavior in its own named, testable class and makes transitions explicit object swaps rather than flag flips. The cost is more classes — overkill for two or three trivial modes.

— How it works

Context	Holds the current ConcreteState and delegates each request to it.
State	The interface declaring the behavior shared by every state.
ConcreteState	Implements one mode's behavior; may trigger the next transition.

HOW TO APPLY

1. Identify the modes and the events that move between them.
2. Define a State interface for the behavior that varies by mode.
3. Put each mode's logic in its own ConcreteState; delegate from the Context.
4. Decide who owns transitions — the state (local) or the Context (central).

LITMUS TEST

Are big conditionals on a status field sprawling across methods, with behavior that depends on mode? If modes keep growing, give each its own state class.

— Smell → fix

One method branches on a status string in every operation:

```
class Order {
  ship() {
    if (this.status === 'paid') doShip()
    else if (this.status === 'new') throw 'unpaid'
  } // every method repeats this ladder
}
```

↓ ONE CLASS PER STATE

```
interface State { ship(o: Order): void }
class Paid implements State {
  ship(o: Order) { doShip(); o.setState(new Shipped()) }
}
class Order {
  state: State = new New()
  ship() { this.state.ship(this) } // delegates
}
```

Each mode's behavior lives in its own class, and Paid.ship() performs the transition to Shipped. Order just delegates to its current state — the switch ladders vanish. For a stateless mode, share one instance as a flyweight.

USE WHEN

- ✓ State machines (order/payment lifecycle)
- ✓ Mode-dependent behavior

AVOID WHEN

- ▲ Few states with trivial branching — an enum/switch is fine

— Trade-offs & pitfalls

PROS

- + Removes big conditionals
- + Transitions are explicit

CONS

- A class per state
- Transition logic can scatter

COMMON PITFALLS

- ▲ Who owns transitions: state-owned couples states to each other; context-owned centralizes the map but fattens the Context — choose deliberately.
- ▲ State explosion: modes times sub-modes multiply classes fast — a transition table or statechart tames it.
- ▲ Logic is distributed across classes, which can hurt the at-a-glance view of the whole machine; two or three trivial modes don't justify it.

WHERE YOU SEE IT

- ▶ Order / payment lifecycles, document workflows, a media player's play / pause / stop.
- ▶ TCP connection states (LISTEN → ESTABLISHED → CLOSED) and CI pipeline stages.
- ▶ Statechart libraries (XState) and React useReducer computing next state from current + action.

SEE ALSO

Strategy	identical UML — but a Strategy is client-set and stays put, State swaps itself
Command	a transition can be reified as a Command, enabling undo of state changes
Memento	snapshot the Context's state to roll a state machine back
Flyweight	share stateless state objects as one immutable instance per mode

— Interview & sources

State vs Strategy?

Same structure; State objects swap themselves as conditions change, a Strategy is chosen once by the client and doesn't self-transition.

Who triggers transitions?

Either the ConcreteState (state-owned) or the Context (context-owned) — a deliberate trade between coupling and centralization.

Why not just a switch?

For a few modes a switch is fine; State pays off when conditionals sprawl across methods and modes keep growing.

Can states be shared?

Yes — if a state keeps no per-context data, reuse one immutable instance as a flyweight singleton.

REMEMBER

One object, many modes — each state is a class that owns its behavior and picks the next state.

SOURCES

GoF — "Design Patterns" (1994): State (behavioral)

Freeman & Robson — "Head First Design Patterns" (2nd ed., 2020): State